

AI Ops Assignment 1

CHIRUTHEJASWI ARAMATI

August 27, 2026

1 Technical Debts Diagnosis

1.1 Entanglement

Changing the rounding of the “estimated delivery time” unexpectedly affected the “favorite restaurants” recommendations.

This is an example of **Entanglement**. It happens when different parts of ML system are connected in ways that we don't expect.

Here, changing the rounding of delivery time looks like a small change, but it also changed the restaurant recommendations. This means that the features are interacting with each other inside the model.

In simple words, **changing one thing changed another thing unexpectedly**.

1.2 Undeclared Consumers

The marketing dashboard was using the raw output table without the ML team's knowledge.

This is an example of **Undeclared Consumers**. It happens when another team or uses some data or output without informing the team that created it.

Here, the marketing team was directly using the ML team's output table. If the ML team changes the table's columns, data format, or update frequency, the marketing dashboard might stop working.

1.3 Pipeline Jungle

The training pipeline had 14 undocumented shell scripts with no orchestration.

This is an example of a **Pipeline Jungle**. It happens when an ML pipeline becomes a collection of many separate scripts without proper organization.

Here, there are 14 shell scripts, but there is no proper documentation showing how they work or depend on each other.

This makes the pipeline:

- difficult to understand
- difficult to test

- difficult to debug
- easy to break when making changes

that means it is something like **too many scripts are doing different jobs without a proper system to manage them.**

2 Mitigation for Pipeline Jungle

A good solution is to replace the 14 shell scripts with a DAG-based orchestration tool such as Airflow, Kubeflow, Dagster, or Prefect.

Instead of having 14 separate scripts running one after another, we can organize them into a proper **DAG (Directed Acyclic Graph)**.

For example:

Data Collection → Data Cleaning → Feature Extraction → Model Training →
Model Evaluation

Each step becomes a separate task in the DAG.

The orchestration tool can then:

- clearly show which task comes before another
- define the input and output of each task
- automatically retry a task if it fails
- give alerts when something goes wrong
- check whether the data is in the correct format
- keep the pipeline definition in version control such as Git

This makes the pipeline much easier to understand and maintain.

In simple words, instead of having 14 confusing scripts, we organize them into a proper flow where every step is clearly connected. This makes the ML pipeline easier to manage, debug, and modify without accidentally breaking the whole system.